Verilog and System Verilog Interview Questions

MAHENDRA AND SATYAM INTERVIEW QUESTION

1. What is Synthesis?
2. Difference between int and integer.
3. Difference between bit [7:0] and byte.
4. Significance of logic.
5. Advantage of dynamic array.
6. Difference between dynamic array, queue and associative array.
7. Difference between verilog and systemverilog.
8. What is static and dynamic?
9. Why can't we use interface instead mailbox?
10. What is callback?
11. What is casting? How it works?
12. Explain polymorphism with an example.
13. What is encapsulation?
14. Question regarding stratified event queue how blocking and non-blocking assignments happens in verilog.
15. Simplify the equation x' + xy.
16. Design xor gate using muxes.
17. What is the use of modports?
18. What is input clock skew and output clock skew?
19. Difference between function and task.
20. How to randomize a variable though it is not declared as rand/randc?
21. How this array works arry [*] [$] [ ].
22. Explain Driver code for RAM.
23. Explain function new and super.build in UVM.
24. Connect virtual interface in driver build_phase.
25. Explain config_db and resource_db.
26. Explain report phase in UVM.
27. features of SV.
28. What is the limitation of randc?
29. Declaring a variable as rand, how to make use of that variable to work as randc.
30. Explain about ignore bins.
31. Explain code coverage.
32. Explain raise objection.
33. Explain global stop request.
34. Difference between Mealy and Moore fsm.
35. Design an fsm for sequence detector 1001.

36. Difference between object and component.
37. Difference between TLM 1.0 and TLM 2.0

Other Questions :

1. A ring counter having 10 pulses .how many flip-flops needed ?
2. How many minimum number of gates required to implement Half adder ?
3. tp =10ns,ts=6ns,th=2ns, calculate clock frequency.

4. Difference between task and function ? option given.

5. Difference between Final and Initial block ? option given.

6. Difference between Latch and Flip-Flops ? option given.

7. Difference between Create() and New() .

8. Find the minimum nu. of transistor required to implement f= A(BC+BDE).Using CMOS
9. FIFO depth calculation.
10. Which gate is required to implement 4:16 decoder using two 3:8 decoder?
a) AND b) XOR c) NOT d) OR e) All of them

11. NAND gate will be .....gate .if it uses -ve logic .

12. Which statement will be execute in the below piece of code

```
casex(3'b110)
        3'bxx1 : statement 1;
        3'bx1x : statement 2;
        3'b1xx : statement 3;
        default : statement 4;
  endcase
```

13.

```
module test;
  reg  x,y ;
  reg [1:0] z;
initial
 begin
  fork
   x =1'b0;
   y =1'b1;
```

```
   z ={y,x};
  join
end
endmodule
```

what should be the value of z in the above piece of code ?

14. In which of the following options callback can be always used

a) Driver
b) Scoreboard
c) Agent
d) Mailbox

16. What is the value of “C” , if q = 1’bx in the below code

```
module test;
reg [1:0] a, b;
     reg  q;
    wire[1:0] c ;
   assign c =(q)?a:b;
  initial  begin
         a=2’b10; b=2’01;
end
endmodule
```

17.
```
always@(posedge clk or negedge reset)
       begin
            if(reset)
            Q <= 1;
            else
            Q<=0 ;
      End
```

Which option is suitable for the above piece of code

a) Asynchronous reset
b) Synchronous reset
c) Asynchronous set
d) Synchronous set

18. Which is not related to the race around condition?

a) Clocking block
b) Program block
c) Non-blocking
d) Mailbox
e) All of the above

19. How many address lines are required to address 256*8 memory?

20. Two Mod 16 counter connected back to back 1st is synchronous & 2nd is ripple counter .

If each flip flop have propagation delay tp then what is the propagation delay of full circuit.

21. Which statement in the following code is wrong?

```
Class abc ;
Integer abc_var ;
endclass
class cde extends abc ;
integer  abc_var   ;
endclass
program  test();
abc   abc_obj ,abc_obj1 ;
cde  cde_obj , cde_obj1 ;
abc_obj1  = new();
cde_obj = abc_obj1 ;
cde_obj1 = new();
abc_obj = cde_obj1;
endprogram
```

22. What type is a Microcontroller?

a) ASIC
b) CPLD
c) FPGA
d) ASSP
e) ISSP

Mobiveil Interview Questions

1) What is functional coverage and code coverage?
2) What is inheritance and polymorphism?
3) Questions related to SVA.
4) Verification flow for VMM and UVM.
5) Difference between OVM and UVM.
6) Questions related to SPI and UART project.
7) Draw verification environment in UVM.
8) Explain the different testcases and scenario to verify the following specification

Consider a block with four 32 bit input channel and four 32 bit output channel, where as input channel are non blocking and each output channel with its own address individually. A packet of data can be forwarded through any one of the input channel or in parallel manner. Packet consist of header, payload and crc bytes.

9) Draw the verification environment in UVM.
10) Write down the code for transaction, driver, monitor for the above scenario.
11) Explain about the scoreboard needs for the above scenario.
12) How will you connect your driver & sequencer and where.
13) What is TLM?
14) Difference between TLM 1.0 and TLM 2.0
15) What is automation in UVM and its purpose?
16) What is configuration database and configuration object.
17) What is overriding in UVM and give some examples.
18) How will you connect your virtual interface in UVM environment.
19) How you will collect data to scoreboard and from where.
20) Explain about VMM and its flow.
21) How you will change static interface to dynamic interface.
22) Difference between for join and begin end.
23) Explain about UDP and FEC coverage.
24) Explain about coverage exclusion.
25) Explain about reusable environment in UVM.
26) Advantages and disadvantages of macros in UVM.

Synopsys Interview Questions

1)Draw the testbench architechture of UVM and Explain.
2)Advantages of UVM over VMM.
3)What is the use of callbacks?
4)Explain a scenario which we need callbacks.
5)What is factory class?
6)How can we override transaction class in UVM and VMM?
7)While overriding how the handle assignment is woking in factory?
8)How to randomize transaction class in sequence?
9)How to find the depth of analysis fifo?
10)Explain about configdb in UVM.
11)How config db is working in UVM?
12)How to map static and virtual interface using config db?
13)Write the code for the above mentioned scenario.

Interview Questions of Sandisk and PerfectVIP Technologies

Written Test

1) Percentage Aptitude.
2) Geometry –triangle Aptitude.
3) Logical reasoning.
4) Sequence detection (2's complement).
5) Cubes Aptitude.
6) Propagation delay.
7) Flip-flop.
8) Mux.
9) FSM.
10) Number systems Aptitude.
11) Analog keyboard switch.
12) Glitches.
13) Setup and hold time.
14) Counters.

Technical Interview

Round-1

1) What is Synchronous and Asynchronous devices and its type.
   Note:- Combinational circuit is asynchronous
2) Different between latch and flip flop.
3) Why do you prefer System Verilog.ie., Advantages of system verilog over verilog.

Round-2

4) Project flow and its details.
5) Body effect and CMOS circuit connections.
6) Types of encoding and which is the more advantage to use.
7) Which one is more preferred in circuit design. NAND or NOR.
   Note:- input capacitance.
8) Types of canonical form, which is preferred.
9) Sensitivity list.
10) Dual port RAM.
11) Propagation delay.
12) Explain Setup and Hold time with waveform example.

i) Violations
ii) How to prevent set and hold time violations.

13) Draw frequency divide by 3 circuit and its waveform.
14) What is inertial delay.
15) Explain callbacks.
16) What is TLM?
17) Draw UVM flow diagram and explain about the different phases.
18) What is the advantage of UVM over System verilog.
19) Determine the output for the given programs which contains function, task and inheritance.
20) Write system verilog verification environment to verify FIFO module.

Specification details:-

Clk_read -10 MHz
Delay - 2 cycle
Total capacity of burst-100 bytes [burst_width]
Clk_write – 15 MHz
Delay – 1cycle
Flags - empty,full

21) TB architecture for SV and explain each blocks and stages.
22) Explain the following terms
    i) Polymorphism
    ii) Inheritance
    iii) Clocking block
    iv) Modport
    v) Interface

Round-3

23) Types of FSM, which type is implemented in your project.
24) Is Ts and Th calculated for combinational circuits and gates.
25) What is meta stability?
26) What is verbosity?

13. What is the difference between mailbox and queue?

Ans:-

Mailbox are FIFO queue, which allows only atomic operations. They can be bounded/unbounded. A bounded mailbox can suspend the thread (while writing if full, while reading if empty) via get/put task. Thats why mailbox is well suited for communication between threads.

24. What is the use of $cast?

Ans:-

Typecasting in SV can be done either via static casting (', ') or dynamic casting via $cast task/function. $cast is very similar to dynamic_cast of C++. It checks whether the casting is possible or not in run-time and errors-out if casting is not possible.

27. What is $unit?

Ans:-

Refer these 2 doc form more details

1. http://www.systemverilog.org/pdf/SystemVerilog_Overall_31A.pdf
2. SV LRM 3.1a :: Section 18.3

28 .What are bi-directional constraints?

Ans:-

Constraints by-default in SystemVerilog are bi-directional. That implies that the constraint solver doesn't follow the sequence in which the constraints are specified. All the variables are looked simultaneously. Even the procedural looking constrains like if ... else ... and -> constrains, both if and else part are tried to solve concurrently. For example (a==0) -> (b==1) shall be solved as all the possible solution of (!(a==0) || (b==1)).

29. What is solve...before constraint ?

Ans:-

In the case where the user want to specify the order in which the constraints solver shall solve the constraints, the user can specify the order via solve before construct. i.e.

```
1....
2.constraint XYZ  {
3.a inside {[0:100]};
4.b < 20;
5.a + b > 30;
6.solve a before b;
7.}
```

The solution of the constraint doesn't change with solve before construct. But the probability of choosing a particular solution change by it.

40. What is circular dependency and how to avoid this problem ?

Ans:-

Over specifying the solving order might result in circular dependency, for which there is no solution, and the constraint solver might give error/warning or no constraining. Example

```
1....
2.int x, y, z;
3.constraint XYZ  {
4.solve x before y;
5.solve y before z;
6.solve z before x;
7.....
8.}
```

4. What is the need of clocking blocks ?

Ans:-

Clocking block in SystemVerilog are used for specifying the clock signal, timing, and synchronization requirements of various blocks. It separates the timing related information from structural, functional and procedural element of the TB. There are quite a few links on clocking block in the internet. These are links to learn about SV clocking blocks.

1. AsicGuru :: To the point answer on the need of clocking block
2. Testbench.in :: Clocking block
3. ProjectVeripage :: Clocking block
4. Doulos :: Clocking block
5. Asicworld :: Clocking block
6. SystemVerilog Event Regions, Race Avoidance & Guidelines

5. What are the ways to avoid race condition between testbench and RTL using SystemVerilog?

Ans:-

Short answer : -

1. Program block
2. Clocking block
3. Enforcement of design signals being driven in non-blocking fashion from program block

Long answer :-

Too long to describe here :). Please refer these doc/sections for more idea/info

1. Section 16.4 of SV LRM
2. http://www.testbench.in/SV_24_PROGRAM_BLOCK.html
3. http://www.sunburst-design.com/papers/CummingsSNUG2006Boston_SystemVerilog_Events.pdf
4. VG discussion of the necessity of program block.

7. What are the types of coverages available in SV ?

Ans:-

Using covergroup : variables, expression, and their cross

Using cover keyword : properties

12. What is the use of the abstract class?

Ans:-

- http://www.testbench.in/CL_08_ABSTRACT_CLASSES.html

(30)Without using randomize method or rand,generate an array of unique values?

Ans:-

```
1....
2.int UniqVal[10];
3.foreach(UniqVal[i]) UniqVal[i] = i;
4.UniqVal.shuffle();
5....
```

(32)What is the difference between byte and bit [7:0]?

Ans:-

byte is signed whereas bit [7:0] is unsigned.

(33)What is the difference between program block and module?

Ans:-

Program block is newly added in SystemVerilog. It serves these purposes

1. It separates testbench from DUT
2. It helps in ensuring that testbench doesn't have any race condition with DUT
3. It provides an entry point for execution of testbench
4. It provides syntactic context (via program ... endprogram) that specifies scheduling in the Reactive Region.

Having said this the major difference between module and program blocks are

1. Program blocks can't have always block inside them, modules can have.
2. Program blocks can't contain UDP, modules, or other instance of program block inside them. Modules don't have any such restrictions.
3. Inside a program block, program variable can only be assigned using blocking assignment and non-program variables can only be assigned using non-blocking assignments. No such restrictions on module
4. Program blocks get executed in the re-active region of scheduling queue, module blocks get executed in the active region
5. A program can call a task or function in modules or other programs. But a module can not call a task or function in a program.

More details:-

1. http://www.testbench.in/SV_24_PROGRAM_BLOCK.html
2. http://www.project-veripage.com/program_blocks_1.php and few more next/next !!!
3. Section 16, SystemVerilog LRM 3.1a ... It's worth the effort reading line-by-line (and between the lines if you can :)).

(37)What is the use of modports?

Ans:-

Modports are part of Interface. Modports are used for specifing the direction of the signals with respect to various modules the interface connects to.

view source
print?

```
1....
2.interface my_intf;
3.wire x, y, z;
4.modport master (input x, y, output z);
5.modport slave  (output x, y, input z);
6.endinterface
```

Please refer section 19.4 of SV LRM for more details

11. Explain about the virtual task and methods .

Ans:-

See http://www.testbench.in/CL_07_POLYMORPHISM.html

(9)What is inheritance and polymorphism?

Please refer these links for more details on inheritance/polymorphism.

1. http://www.testbench.in/CL_00_INDEX.html
2. SV OOP Links

(14)What data structure you used to build scoreboard?

Ans:-

Queue

(16)How parallel case and full cases problems are avoided in SV ?

Ans:-

See Page 34/35 of http://www.systemverilog.org/pdf/SV_Symposium_2003.pdf

(22)What is the use of package?

Ans:-

In Verilog declaration of data/task/function within modules are specific to the module only. They can't be shared between two modules. Agreed, we can achieve the same via cross module referencing or by including the files, both of which are known to be not a great solution.

The package construct of SystemVerilog aims in solving the above issue. It allows having global data/task/function declaration which can be used across modules. It can contain module/class/function/task/constraints/covergroup and many more declarations (for complete list please refer section 18.2 of SV LRM 3.1a)

The content inside the package can be accessed using either scope resolution operator (**::**), or using import (with option of referencing particular or all content of the package).

view source
print?

```
01.package ABC;
02.// Some typedef
03.typedef enum {RED, GREEN, YELLOW} Color;
04.
05.// Some function
06.void function do_nothing()
07....
08.endfunction : do_nothing
09.
10.// You can have many different declarations here
11.endpackage : ABC
12.
13.// How to use them
14.import ABC::Color;     // Just import Color
15.import ABC::*;         // Import everything inside the package
```

(26)What is $root?

Ans:-

$root refers to the top level instance in SystemVerilog

```
1.package ABC;
2.$root.A;       // top level instance A
3.$root.A.B.C;   // item C within instance B within top level instance A
```

(25)What is the difference between rand and randc?

Ans:-

rand - Random Variable, same value might come before all the the possible value have been returned. Analogous to throwing a dice.

randc - Random Cyclic Variable, same value doesn't get returned until all possible value have been returned. Analogous to picking of card from a deck of card without replacing. Resource intensive, use sparingly/judiciously

(24)How to call the task which is defined in parent object into derived class ?

Ans:-

super.task_name();

(21)What is the difference between always_comb and always@(*)?

Ans:-

From SystemVerilog LRM 3.1a:-

1. always_comb get executed once at time 0, always @* waits till a change occurs on a signal in the inferred sensitivity list
2. Statement within always_comb can't have blocking timing, event control, or fork-join statement. No such restriction of always @*
3. Optionally EDA tool might perform additional checks to warn if the behavior within always_comb procedure doesn't represent combinatorial logic
4. Variables on the left-hand side of assignments within an always_comb procedure, including variables from the contents of a called function, shall not be written to by any other processes, whereas always @* permits multiple processes to write to the same variable.
5. always_comb is sensitive to changes within content of a function, whereas always @* is only sensitive to changes to the arguments to the function.

A small SystemVerilog code snippet to illustrate #5

```
01.module dummy;
02.logic a, b, c, x, y;
03.
04.// Example void function
05.function void my_xor;
06.input a;            // b and c are hidden input here
07.x = a ^ b ^ c;
08.endfunction : my_xor
09.
10.function void my_or;
11.input a;            // b and c are hidden input here
12.y = a | b | c;
13.endfunction : my_xor
14.
15.always_comb     // equivalent to always(a,b,c)
16.my_xor(a);          // Hidden inputs are also added to sensitivity list
17.
18.always @*              // equivalent to always(a)
19.my_or(a);           // b and c are not added to sensitivity list
20.endmodule
```

(20)List the predefined randomization methods.

Ans:-

1. randomize
2. pre_randomize
3. post_randomize

(19)What is scope randomization ?

Ans:-

Scope randomization ins SystemVerilog allows assignment of unconstrained or constrained random value to the variable within current scope

view source
print?

```
01.module MyModule;
```

```
02.integer var, MIN;

03.
04.initial begin

05.MIN = 50;

06.for ( int i = 0;i<10 ;i++)
 begin
07.if( randomize(var) with { var < 100 ; var > MIN ;})

08.$display(" Randomization sucsessfull : var = %0d Min = %0d",var,MIN);

09.else

10.$display("Randomization failed");

11.end
12.
13.$finish;

14.end
15.endmodule
```

(88)How to check weather a handles is holding object or not ?

Ans:-

It is basically checking if the object is initialized or not. In SystemVerilog all uninitialized object handles have a special value of null, and therefore whether it is holding an object or not can be found out by comparing the object handle to null. So the code will look like:-

```
01.usb_packet My_usb_packet;
02....
03.if(My_usb_packet == null) begin
04.// This loop will get exited if the handle is not holding any object
05.....
06.end else begin
07.// Hurray ... the handle is holding an object
08....
09.end
```

(87)What is the difference between initial block and final block?

Ans:-

There are many difference between initial and final block. I am listing the few differences that is coming to mind now.

1. The most obvious one : Initial blocks get executed at the beginning of the simulation, final block at the end of simulation
2. Final block has to be executed in zero time, which implies it can't have any delay, wait, or non-blocking assignments. Initial block doesn't have any such restrictions of execution in zero time (and can have delay, wait and non-blocking statements)

Final block can be used to display statistical/genaral information regarding the status of the execution like this:-

```
1.final begin
2.$display("Simulation Passed");
3.$display("Final value of xyz = %h",xyz);
4.$display("Bye :: So long, and Thanks for all the fishes");
5.end
```

(69)What is the difference between bits and logic?
Ans:-
bits is 2-valued (1/0) and logic is 4-valued (0/1/x/z)

(65)What is tagged union ?
Ans:-
An union is used to stored multiple different kind/size of data in the same storage location.

```
1.typedef union{
2.bit [31:0]  a;
3.int    b;
4.} data_u;
```

Now here XYZ union can contain either bit [31:0] data or an int data. It can be written with a bit [31:0] data and read-back with a int data. There is no type-checking done.

In the case where we want to enforce that the read-back data-type is same as the written data-type we can use tagged union which is declared using the qualifier **tagged**. Whenever an union is defined as tagged, it stores the tag information along with the value (in expense of few extra bits). The tag and values can only be updated together using a statically type-checked tagged union expression. The data member value can be read with a type that is consistent with current tag value, making it impossible to write one type and read another type of value in tagged union. (the details of which can be found in section 3.10 and 7.15 of SV LRM 3.1a).

view source
print?

```
01.typedef union tagged{
02.bit [31:0]  a;
03.int    b;
04.} data_tagged_u;
05.
06.// Tagged union expression
07.data_tagged_u data1 = tagged a 32'h0;
08.data_tagged_u data2 = tagged b 5;
09.
10.// Reading back the value
11.int xyz = data2.b;
```

(56)What is the need of alias in SV?
Ans:-
The Verilog has one-way assign statement is a unidirectional assignment and can contain delay and strength change. To have bidirectional short-circuit connection SystemVerilog has added **alias** statement. An excellent usage example of alias can be found out at http://www.systemverilog.org/pdf/SV_Symposium_2003.pdf(Slide # 59)

Wire:-

1. Wires are used for connecting different elements
2. They can be treated as a physical wire
3. They can be read or assigned
4. No values get stored in them
5. They need to be driven by either continuous assign statement or from a port of a module

Reg:-

1. Contrary to their name, regs doesn't necessarily corresponds to physical registers
2. They represents data storage elements in Verilog/SystemVerilog
3. They retain their value till next value is assigned to them (not through assign statement)
4. They can be synthesized to FF, latch or combinational circuit (They might not be synthesizable !!!)

Wires and Regs are present from Verilog timeframe. SystemVerilog added a new data type called logic to them. So the next question is what is this logic data type and how it is different from our good old wire/reg.

Logic:-

1. As we have seen, reg data type is bit mis-leading in Verilog. SystemVerilog's logic data type addition is to remove the above confusion. The idea behind having a new data type called logic which at least doesn't give an impression that it is hardware synthesizable
2. Logic data type doesn't permit multiple driver. It has a last assignment wins behavior in case of multiple assignment (which implies it has no hardware equivalence). Reg/Wire data type give X if multiple driver try to drive them with different value. Logic data type simply assign the last assignment value.
3. The next difference between reg/wire and logic is that logic can be both driven by assign block, output of a port and inside a procedural block like this

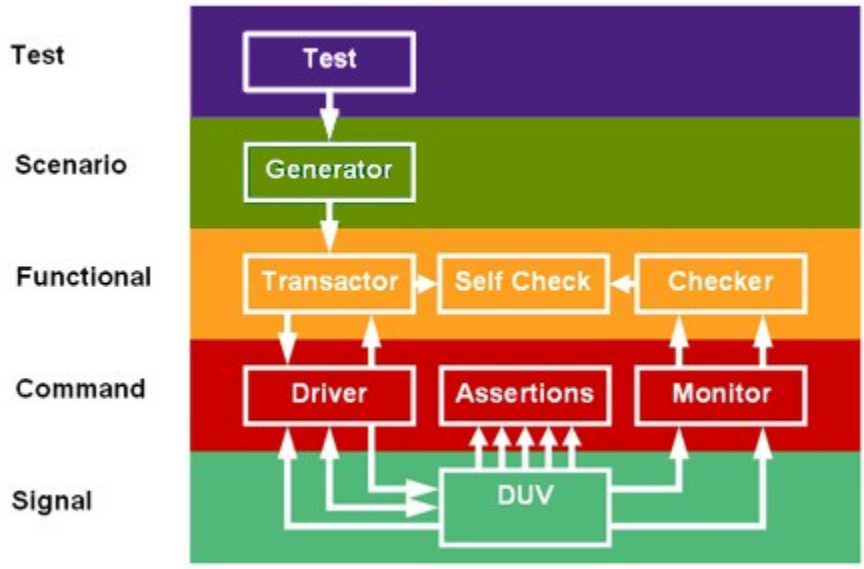
```
logic a;
assign a = b ^ c;                    // wire style
always (c or d) a = c + d;           // reg style
MyModule module(.out(a), .in(xyz)); // wire style
```

(86) What is the use of "extern"?
(66)What is "scope resolution operator"?

Ans:-
extern keyword allows out-of-body method declaration in classes. Scope resolution operator (::) links method declaration to class declaration.

```
class XYZ;
// SayHello() will be declared outside the body
// of the class
extern void task SayHello();
endclass : XYZ

void task XYZ :: SayHello();
$Message("Hello !!!\n");
endtask : SayHello
```



(76) What is layered architecture ?

Ans:-
In SystemVerilog based constrained random verification environment, the test environment is divided into multiple layered as shown in the figure. It allows verification component re-use across verification projects.

(71)What is the difference between $rose and posedge?

Ans:-
posedge return an event, whereas $rose returns a Boolean value. Therefore they are not interchangeable.

(64)What is "this"?

Ans:-
"this" pointer refers to current instance.

(38)Write a clock generator without using always block.

Ans:-

```
initial begin
   clk <= '0;
   forever #(CYCLE/2) clk = ~clk
end
```

(35)How to implement always block logic in program block ?
Ans:-

Use of forever begin end. If it is a complex always block statement like always (@ posedge clk or negedge reset_)

```
always @(posedge clk or negedge reset_) begin
   if(!reset_) begin
       data <= '0;
   end else begin
       data <= data_next;
```

```
   end
end

// Using forever : slightly complex but doable
forever begin
   fork
   begin : reset_logic
       @ (negedge reset_);
       data <= '0;
   end : reset_logic
   begin : clk_logic
       @ (posedge clk);
       if(!reset_)     data <= '0;
       else            data <= data_next;
   end : clk_logic
   join_any
   disable fork
end
```

10> **What is the need of virtual interface ?**

Ans:-

An interface encapsulate a group of inter-related wires, along with their directions (via modports) and synchronization details (via clocking block). The major usage of interface is to simplify the connection between modules.

But Interface can't be instantiated inside program block, class (or similar non-module entity in SystemVerilog). But they needed to be driven from verification environment like class. To solve this issue virtual interface concept was introduced in SV.

Virtual interface is a data type (that implies it can be instantiated in a class) which hold reference to an interface (that implies the class can drive the interface using the virtual interface). It provides a mechanism for separating abstract models and test programs from the actual signals that make up the design. Another big advantage of virtual interface is that class can dynamically connect to different physical interfaces in run time.

For more details please refer the following links

- Testbench.in Interface tutorial
- Asicworld.com Interface tutorial

18> **What is difference between $random() and $urandom()**

Ans:-

1. $random system function returns a 32-bit signed random number each time it is called
2. $urandom system function returns a 32-bit unsigned random number each time it is called. (newly added in SV, not present in verilog)

47> **How to randomize dynamic arrays of an object**

Ans:-

```
class ABC;
// Dynamic array
rand bit [7:0] data [];

// Constraints
constraint cc {
// Constraining size
data.size inside {[1:10]};

// Constraining individual entry
data[0] > 5;

// All elements
foreach(data[i])
if(i > 0)
data[i] > data[i-1];
}
endclass : ABC
```

output of the following block

```
always @(posedge clk) begin
   a = 0;
   a <= 1;
   $display(s);
end
```

To begin with let me be clear that this is not the correct way to code in verilog/SV. One shouldn't mix blocking and non-blocking assignment in the same begin-end block. But this question is asked to check the knowledge of scheduling semantics of verilog/SV.

Verilog scheduling semantics basically imply a four-level deep queue for the current simulation time:-

1. Active Events (blocking assignment, RHS of NBA, continuous assignment, $display, ...)
2. Inactive Events (#0 blocking assignment)
3. Non-Blocking Assign Updates (LHS of NBA)
4. Monitor Events ($monitor, $strobe).

Which implies, a=0 will be get printed in this example.

System Verilog has added few more level queue for simulation, the details of which can be found in these 2 excellent link on scheduling semantics of SV.

- SystemVerilog 3.1 scheduling semantics
- SystemVerilog Event Regions, Race Avoidance & Guidelines

System Verilog Interview questions from http://www.edaboard.com/ftopic315416.html. I don't know answers to all of the questions, but will try to find out their answer from internet (thanks to the all powerful Google) and post the answers in this blog (one by one most likely). If you know answers to any of these then you are well come to share the same by commenting to this post.

1. What is callback ?
2. What is factory pattern ?
3. Explain the difference between data types logic and reg and wire
4. What is the need of clocking blocks ?
5. What are the ways to avoid race condition between testbench and RTL using SystemVerilog?
6. Explain Event regions in SV.
7. What are the types of coverages available in SV ?
8. What is OOPS?
9. What is inheritance and polymorphism?
10. What is the need of virtual interfaces ?
11. Explain about the virtual task and methods .
12. What is the use of the abstract class?
13. What is the difference between mailbox and queue?
14. What data structure you used to build scoreboard
15. What are the advantages of linkedlist over the queue ?
16. How parallel case and full cases problems are avoided in SV
17. What is the difference between pure function and cordinary function ?
18. What is the difference between $random and $urandom?
19. What is scope randomization
20. List the predefined randomization methods.
21. What is the dfference between always_combo and always@(*)?
22. What is the use of packagess?
23. What is the use of $cast?
24. How to call the task which is defined in parent object into derived class ?
25. What is the difference between rand and randc?
26. What is $root?
27. What is $unit?
28. What are bi-directional constraints?
29. What is solve...before constraint ?
30. Without using randomize method or rand,generate an array of unique values?
31. Explain about pass by ref and pass by value?
32. What is the difference between bit[7:0] sig_1; and byte sig_2;
33. What is the difference between program block and module ?
34. What is final block ?
35. How to implement always block logic in program block ?
36. What is the difference between fork/joins, fork/join_none fork/join_any ?
37. What is the use of modports ?
38. Write a clock generator without using always block.
39. What is forward referencing and how to avoid this problem?
40. What is circular dependency and how to avoid this problem ?
41. What is cross coverage ?
42. Describe the difference between Code Coverage and Functional Coverage Which is more important and Why we need them

43. How to kill a process in fork/join?
44. Difference between Associative array and Dynamic array ?
45. Difference b/w Procedural and Concurrent Assertions?
46. What are the advantages of SystemVerilog DPI?
47. How to randomize dynamic arrays of objects?
48. What is randsequence and what is its use?
49. What is bin?
50. Why always block is not allowed in program block?
51. Which is best to use to model transaction? Struct or class ?
52. How SV is more random stable then Verilog?
53. Difference between assert and expect statements?
54. How to add a new processs with out disturbing the random number generator state ?
55. What is the need of alias in SV?
56. What is the need to implement explicitly a copy() method inside a transaction , when we can simple assign one object to other ?
57. How different is the implementation of a struct and union in SV.
58. What is "this"?
59. What is tagged union ?
60. What is "scope resolution operator"?
61. What is the difference between Verilog Parameterized Macros and SystemVerilog Parameterized Macros?
62. What is the difference between

```
1.logic data_1;
2.var logic data_2;
3.wire logic data_3j;
4.bit data_4;
5.var bit data_5;
```

63. What is the difference between bits and logic?
64. Write a Statemechine in SV styles.
65. What is the difference between $rose and posedge?
66. What is advantage of program block over clockcblock w.r.t race condition?
67. How to avoid the race condition between programblock ?
68. What is the difference between assumes and assert?
69. What is coverage driven verification?
70. What is layered architecture ?
71. What are the simulation phases in your verification environment?
72. How to pick a element which is in queue from random index?
73. What data structure is used to store data in your environment and why ?
74. What is casting? Explain about the various types of casting available in SV.
75. How to import all the items declared inside a package ?
76. Explain how the timescale unit and precision are taken when a module does not have any timescalerdeclaration in RTL?
77. What is streaming operator and what is its use?
78. What are void functions ?
79. How to make sure that a function argument passed has ref is not changed by the function?
80. What is the use of "extern"?

81. What is the difference between initial block and final block?
82. How to check weather a handles is holding object or not ?
83. How to disable multiple threads which are spawned by fork...join